arXiv:2306.02561v3 [cs.CL] 30 Jun 2023

LLM-BLENDER: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion

Dongfu Jiang[♡] **Xiang Ren**[♣♠] **Bill Yuchen Lin**[♠]
dongfu@zju.edu.cn, xiangren@usc.edu, yuchenl@allenai.org
[♠]Allen Institute for Artificial Intelligence
[♣]University of Southern California [♡]Zhejiang University

Abstract

We present LLM-BLENDER, an ensembling framework designed to attain consistently superior performance by leveraging the diverse strengths of multiple open-source large language models (LLMs). Our framework consists of two modules: PAIRRANKER and GENFUSER, addressing the observation that optimal LLMs for different examples can significantly vary. PAIRRANKER employs a specialized pairwise comparison method to distinguish subtle differences between candidate outputs. It jointly encodes the input text and a pair of candidates, using cross-attention encoders to determine the superior one. Our results demonstrate that PAIRRANKER exhibits the highest correlation with ChatGPT-based ranking. Then, GENFUSER aims to merge the top-ranked candidates, generating an improved output by capitalizing on their strengths and mitigating their weaknesses. To facilitate large-scale evaluation, we introduce a benchmark dataset, MixInstruct, which is a mixture of multiple instruction datasets featuring oracle pairwise comparisons. Our LLM-BLENDER significantly outperform individual LLMs and baseline methods across various metrics, establishing a substantial performance gap. [1 2]

1 Introduction

Large language models (LLMs) have shown impressive performance in diverse tasks, primarily due to their capacity to follow instructions and access extensive, high-quality data, showing a promising future for artificial general intelligence (Bubeck et al., 2023). However, prominent LLMs such as GPT-4 and PaLM (Chowdhery et al., 2022) are closed-source, restricting insights into their architectures and training data. Open-source LLMs like Pythia (Biderman et al., 2023), LLaMA (Touvron et al., 2023), and Flan-T5 (Chung et al., 2022) offer a chance to fine-tune these models on custom instruction datasets, enabling the development of smaller yet efficient LLMs, such as Alpaca, Vicuna (Chiang et al., 2023), OpenAssistant (LAION-AI, 2023), and MPT (MosaicML, 2023).

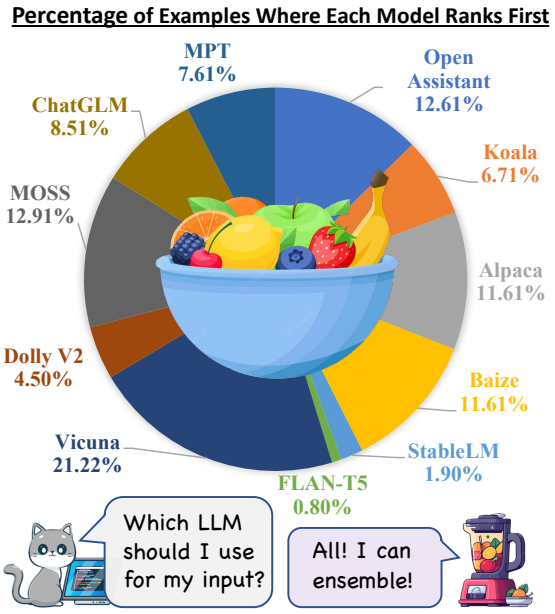

Figure 1: **Motivation of ensembling LLMs.** Based on this pie chart about the percentage of examples where each LLM ranks 1st, we can see that optimal LLMs for different examples can significantly vary.

The open-source LLMs exhibit diverse strengths and weaknesses due to variations in data, architectures, and hyperparameters, making them complementary to each other. Figure 1 illustrates the distribution of best LLMs on 5,000 instructions that we collected. More ranking details can be found in Sec. 5.1. Although Vicuna achieves the highest percentage, it ranks first in only 21.22% of the examples. Furthermore, the pie chart suggests that the optimal LLMs for different examples can significantly vary and there is no open-source LLM

[1] https://yuchenlin.xyz/LLM-Blender

[2] The experiments on summarization, translation, and constrained generation tasks in the prior version have been moved to the appendix. Instead, we mainly present our work in the context of instruction-following data and LLMs in this version.

that dominates the competition. Therefore, it is important to dynamically ensemble these LLMs to generate consistently better responses for each input. Considering the diverse strengths and weaknesses of LLMs, it is crucial to develop an ensembling method that harnesses their complementary potentials, leading to improved robustness, generalization, and accuracy. By combining their unique contributions, we can alleviate biases, errors, and uncertainties in individual LLMs, resulting in outputs better aligned with human preferences.

We introduce **LLM-BLENDER**, an ensembling framework designed to achieve consistently superior performance by mixing the outputs of multiple LLMs. LLM-BLENDER comprises two modules: PAIRRANKER and GENFUSER. Initially, PAIRRANKER compares the outputs from N LLMs, which GENFUSER then fuses to generate the final output from the top K ranked outputs.

Existing approaches (Ravaut et al., 2022a; Liu and Liu, 2021), including the reward model within InstructGPT (Ouyang et al., 2022), for ranking outputs $\{y_1, \ldots, y_N\}$ from language models (LMs) on a given input x have mostly focused on *individually scoring* each y_i based on x, employing encoding modules in the form of $s_i = f_\phi(x, y_i)$. Although this list-wise ranking objective can be powerful and efficient when candidate differences are apparent, it may not be as effective when ensembling LLMs. Among the output candidates from LLMs, candidate differences can be quite *subtle*, as they are all produced by very sophisticated models and one may only be marginally better than another. Even for humans, it can be challenging to gauge candidate quality without direct comparison.

As a result, we propose a specialized *pairwise* comparison method, **PAIRRANKER** (Sec. 3), to effectively discern subtle differences between candidate outputs and enhance ranking performance. In particular, we first gather the outputs from N models (e.g., the $N = 11$ models in Fig. 1) for each input and subsequently create the $N(N-1)/2$ pairs of their outputs. We jointly encode the input x and the two candidate outputs y_i and y_j as input to a cross-attention encoder (e.g., RoBERTa (Liu et al., 2019)), in the form of $f_\phi(x, y_i, y_j)$, to learn and determine which candidate is better.

During the inference stage, we compute a matrix containing logits representing pairwise comparison results. Given this matrix, we can infer a ranking of the N outputs for the given input x. Subsequently, we can employ the top-ranked candidate from PAIRRANKER for each input as the final result. Hence, this approach does not rely on a single model for all examples; instead, PAIRRANKER selects the best model for each example by comprehensively comparing all candidate pairs.

Nonetheless, this approach may constrain the potential to generate even better outputs than the existing candidates. To investigate this possibility, we introduce the **GENFUSER** (Sec. 4) module to fuse the top K of the N ranked candidates and generate an improved output for end-users. Our goal is to capitalize on the strengths of the top K selected candidates while mitigating their weaknesses.

To assess the effectiveness of LLM ensembling methods, we introduce a benchmark dataset called **MixInstruct** (Sec. 2.2). In this dataset, we use N=11 popular open-source LLMs to generate N candidates for each input across various existing instruction-following tasks formatted as self-instruct (Wang et al., 2022). The dataset comprises 100k training examples and 5k validation examples for training a candidate ranking module like our PAIRRANKER, and 5k test examples with oracle comparisons for automatic evaluation.

In Section 5, our empirical results on the MixInstruct benchmark reveal that the LLM-BLENDER framework significantly boosts overall performance by ensembling LLMs. The selections made by PAIRRANKER outperform any fixed individual LLM models, as indicated by superior performance in both reference-based metrics and GPT-Rank. By leveraging the top selections from PAIRRANKER, GENFUSER further enhances response quality through effective fusion into the final output. LLM-BLENDER achieves the highest scores in terms of both conventional metrics (i.e., BERTScore, BARTScore, BLUERT) and ChatGPT-based ranking. The average rank of LLM-BLENDER stands at 3.2 among the 12 methods, which is considerably better than the best LLM's rank of 3.90. Moreover, LLM-BLENDER's output ranks in the top 3 for 68.59% of examples, while Viccuna only reaches 52.88%. We believe LLM-BLENDER and our findings would benefit both practitioners and researchers for deploying and studying LLMs with ensemble learning.

2 Preliminaries

We first provide the problem formulation and two common types of ensembling methods. Next, we

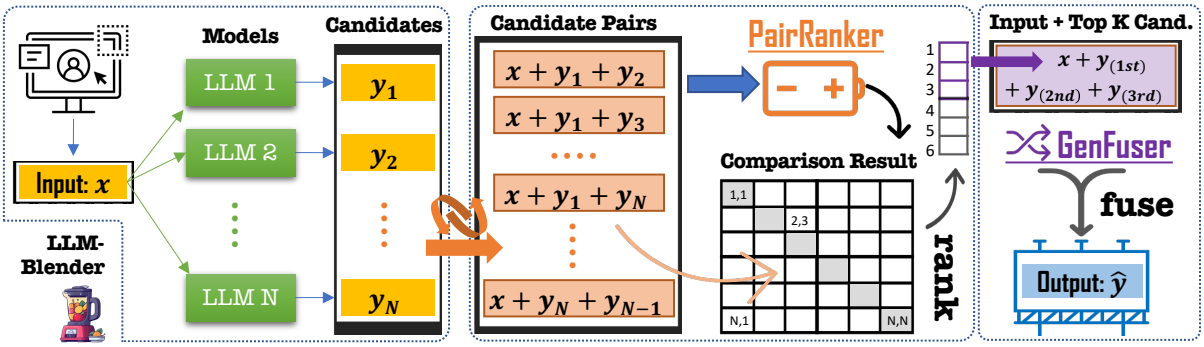

Figure 2: **The LLM-BLENDER framework.** For each input x from users, we employ N different LLMs to get output candidates. Then, we pair all candidates and concatenate them with the input before feeding them to PAIRRANKER, producing a matrix as comparison results. By aggregating the results in the matrix, we can then rank all candidates and take the top K of them for generative fusion. The GENFUSER module concatenates the input x with the K top-ranked candidates as input and generate the final output $\hat{y}$.

present the dataset `MixInstruct` created for training and evaluation purposes. Finally, we give an overview of our framework.

2.1 Problem Setup

Given an input x and N models, $\{\mathcal{M}_1, \ldots, \mathcal{M}_N\}$, we can generate N candidate outputs by processing x with each model. We denote the candidates as $\mathbb{Y} = \{y_1, \ldots, y_N\}$. In the training data, we assume there is a ground truth output, y, while it remains hidden during evaluation at test time.

In practice, one might choose a fixed model, such as $\mathcal{M}_9$, to infer all unseen examples (i.e., always using y_9 as the final output for x). This can be reasonable if $\mathcal{M}_9$ demonstrates significantly better overall performance on certain observed examples. However, relying on a pre-selected model may result in sub-optimal performance, as the N models likely possess different strengths and weaknesses in various situations, meaning that the optimal selection for different x values may not always originate from the same model.

Our objective is to develop an ensemble learning method that produces an output $\hat{y}$ for the input x, maximizing the similarity $Q(\hat{y}, y; x)$. The Q function can be implemented in various ways, which we will discuss later. We anticipate that this method will yield better overall performance than using a fixed model or randomly selecting a model for x. Specifically, given a test set $D_{\text{test}} = \{(x^{(i)}, y^{(i)})\}$, we aim to maximize $\sum_i Q(\hat{y}^{(i)}, y^{(i)}; x^{(i)})$.

There are two primary approaches for ensembling LLMs: *selection-based* and *generation-based* methods. Selection-based methods compare candidates in the set $\mathbb{Y}$, selecting the top-ranked candidate as the final output $\hat{y}$, which implies that $\hat{y} \in \mathbb{Y}$. Due to the inherent nature of selection and the limited solution space, the performance of selection-based methods is bounded by the N candidates being considered. Conversely, generation-based methods focus on fusing K candidates ($1 < K \leq N$) from $\mathbb{Y}$ to produce an unseen response as the final output $\hat{y}$.

Sources	#Examples	Source	I/O Tokens
Alpaca-GPT4	22,862	GPT-4	22 / 48
Dolly-15K	7,584	Human	24 / 53
GPT4All-LAION	76,552	ChatGPT	18 / 72
ShareGPT	3,002	ChatGPT	36 / 63
Total	110K	Mix	20 / 66

Table 1: **Statistics of `MixInstruct`.** It contains 110K examples and we randomly split the dataset into train/dev/test in 100K/5K/5K sizes.

2.2 `MixInstruct`: A New Benchmark

We introduce a new dataset, `MixInstruct`, to benchmark ensemble models for LLMs in instruction-following tasks. We collect a large-scale set of instruction examples primarily from four sources, as shown in Table 1. After curating and processing this open-source data, we sample 100k examples for training, 5k for validation, and 5k for testing. We then run $N = 11$ popular open-source LLMs, including Vicuna, OpenAssistant, Alpaca, MPT, and others (see Table 2 and Figure 1), on these 110k examples.

To obtain the oracle ranking of candidates, we design comparative prompts for ChatGPT to evaluate all candidate pairs. Specifically, for each exam-

ple, we prepare 55 pairs of candidates ($11 \times 10/2$). For each pair, we ask ChatGPT to judge the better candidate (or declare a tie). The prompt template can be found in the appendix. For the training and validation sets, we provide the results based on conventional metrics like BERTScore, BLEURT, and BARTScore. In that case, we use function $Q(y_i, y)$ to estimate a candidate y_i's quality according to its similarity to the ground truth y.

2.3 LLM-BLENDER: A Novel Framework

We propose a rank-and-fuse pipeline framework, LLM-BLENDER, for ensembling LLMs, as illustrated in Figure 2. This framework consists of two main components: a pairwise ranking module, PAIRRANKER (Section 3), and a fusion module, GENFUSER (Section 4). The PAIRRANKER module learns to compare all pairs of candidates for each input and subsequently rank the list of candidates. We then select the top $K = 3$ ranked candidates, concatenate them with the input x, and construct the input sequence for the GENFUSER module. The GENFUSER module, a seq2seq LM, ultimately generates the final output to serve users.

3 PAIRRANKER: Pairwise Ranking

In this section, we introduce three baseline methods for ranking the candidates in $\mathbb{Y}$ in Sec. 3.1 and present the proposed PAIRRANKER method.

3.1 Baseline Methods

Previous reranking methods primarily focus on computing the score $s_i = f_\phi(x, y_i)$ for each candidate $y_i \in \mathbb{Y}$ independently, where s_i is solely determined by y_i. Notably, the reward model in instruction tuning for GPT-3.5 (Ouyang et al., 2022) also belongs to this category. Figure 3 illustrates these baseline methods, which are further detailed in the following paragraphs.

MLM-Scoring (Salazar et al., 2020) assesses the quality of a candidate by calculating its pseudo-log-likelihood, which is obtained by masking tokens one by one and computing the log-likelihood for the masked token using masked LMs (e.g., BERT). Given a candidate y_i as a sequence of words $\mathbf{W} = \{w_1, ..., w_{|\mathbf{W}|}\}$, the pseudo-log-likelihood is: $s_i = \sum_{t=1}^{|\mathbf{W}|} \log P(w_t|\mathbf{W}_{\setminus t})$. This unsupervised method is effective for reranking outputs in NLG tasks such as machine translation and speech recognition.

SimCLS (Liu and Liu, 2021) encodes the input x and each generated candidate $y_i \in \mathbb{Y}$ using the same encoder H, resulting in $H(x)$ and $H(y_i)$. The cosine similarity between them, $s_i = \cos(H(x), H(y_i))$, serves as the predicted score, as $H(x)$ and $H(y_i)$ share the same embedding space induced by the language encoder. In training, marginal ranking loss is used to optimize H.

SummaReranker (Ravaut et al., 2022a) concatenates the input x and each candidate y_i, using a cross-attention encoder to learn ranking. Specifically, they employ $H([x; y_i])$ to predict the score s_i, where H is a Transformer model. In the training stage, binary cross-entropy (BCE) loss is employed to differentiate the best candidate from the others.

Limitations. Despite using contrastive loss in training, these methods rely on individual scoring for inference. The encoders have not been exposed to pairs of candidates for direct comparison learning. We argue that such pointwise ranking methods may be insufficient for selecting the best candidates in the context of LLMs and instruction-following tasks. One reason is that the quality of LLM outputs is generally high when the chosen LLMs are popular and competitive. Moreover, the responses for instruction tasks can be quite open-ended, unlike summarization tasks. Therefore, merely examining individual candidates may not yield a reliable score. This issue becomes more prominent for shorter responses, where sequences may differ by only a few words but vary significantly in helpfulness, harmfulness, and fairness. Given these limitations, we contend that individual scoring approaches may fail to capture crucial nuances.

3.2 Pairwise Comparisons

In order to address the limitations of pointwise ranking, we aim to train a ranker f with parameter ϕ that can compare a pair of output candidates by encoding them together with the input text. Our ranker module should focus on learning to capture the differences between the two candidates and prefer the ones of higher quality.

Given a pair of candidates y_i, y_j, we obtain their pair-specific scores: $s^i_{(i,j)}$ and $s^j_{(i,j)}$. We denote the model's confidence in thinking y_i is better than y_j as $s_{ij} = s^i_{(i,j)} - s^j_{(i,j)}$. We can use these scores for all pairs induced from $\mathbb{Y}$ to infer the final ranking. To learn this ability, we concatenate the input x and the two candidates to form a sequence $[x; y_i; y_j]$ and feed it into a cross-attention Transformer to get the features: $f_\phi([x; y_i; y_j])$ for modeling s_{ij}.

We assume multiple Q functions to optimize

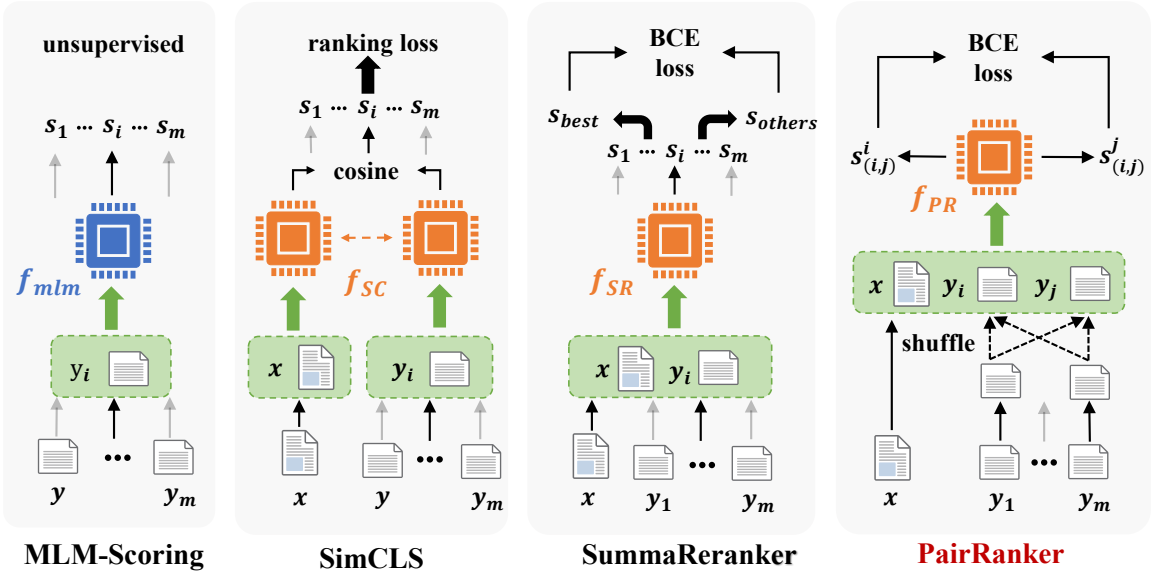

Figure 3: **The architectures of typical reranking methods.** x is an input and y_i is a certain candidate, and its score is s_i. MLM-Scoring is an unsupervised method that uses an external masked LM to score a candidate; SimCLS uses the same encoder to encode x and each candidate y_i; SummaReranker instead employs a cross-encoder to encode both x and y_i at the same time; PairRanker encodes a pair of candidates (y_i, y_j) at the same time for pairwisely scoring them, and the final score of each candidate is produced as shown in Fig. 4.

for, such as BERTScore, BARTScore, etc., and consider the learning problem as a multi-task classification problem:

$$\mathcal{L}_Q = -z_i \log \sigma(s^i_{(i,j)}) - z_j \log \sigma(s^j_{(i,j)}),$$

where σ denotes the sigmoid function and

$$(z_i, z_j) = \begin{cases} (1,0), & Q(y_i, y) \geq Q(y_j, y) \\ (0,1), & Q(y_i, y) < Q(y_j, y) \end{cases}.$$

For optimizing towards multiple Q, we take the average as the final multi-objective loss: $\mathcal{L} = \sum \mathcal{L}_Q$.

3.3 PairRanker Architecture

We discuss the concrete designs for the PairRanker module in this subsection.

Encoding. We employ Transformer layers to encode an input and a pair of candidates, enabling the attentions to capture the difference between candidates in the context of the input. We concatenate the three segments sequentially and form a single input sequence with special tokens as separators: `<source>`, `<candidate1>`, and `<candidate2>`. The resulting input sequences to Transformers are in the form of "`<s><source>` x `</s>` `<candidate1>` y_i `</s>` `<candidate2>` y_j `</s>`", where x is the text of a source input and y_i and y_j are the text of two output candidates. The embeddings of special tokens `<source>`, `<candidate1>`, and `<candidate2>` are used as the representations of x, y_i, and y_j respectively, denoted as $\mathbf{x}$, $\mathbf{y_i}$, $\mathbf{y_j}$.

Training. To determine the scores for the two candidates, we concatenate the embeddings of $\mathbf{x}$ with $\mathbf{y_i}$ and $\mathbf{y_j}$ respectively, and pass them through a single-head layer, which is a multi-layer perceptron with the final layer's dimension equal to the number of Q functions to be optimized. Each value within this dimension represents a computed Q score for a specific Q function. We derive the final score $s^i_{(i,j)}$ or $s^j_{(i,j)}$ for the candidate by averaging these Q scores. Since there are $O(N^2)$ unique pair combinations, we apply an effective sub-sampling strategy during the training stage to ensure learning efficiency.

During training, we randomly select some combinations from the candidate pool $\mathbb{Y}^2$, instead of all the $N(N-1)/2$ pairs. We also compare the target text with other candidates by extending the candidate pool by mixing the ground truth y into $\mathbb{Y}$. In practice, we found that using 5 pairs per input is sufficient for obtaining decent results.

Due to the position embeddings of the language model, the order of the candidates in a pair (x, y_i, y_j) matters, as the comparison result of

(x, y_i, y_j) and (x, y_j, y_i) might not be consistent. Thus, we shuffle the order of candidates within each training pair so that the model learns to be consistent with itself.

Inference. During the inference stage, we obtain scores s_{ij} for each pair of candidates $(y_i, y_j) \in \mathbb{Y}^2$. After $N(N-1)$ iterations, we obtain a matrix $\mathbf{M}$, where $\mathbf{M}_i^j = s_{ij}$ represents the *confidence* that y_i is better than y_j. To identify the best candidate based on $\mathbf{M}$, we introduce three aggregation functions for determining the final ranking of $\mathbb{Y}$.

We propose two scoring methods, `MaxLogits` and `MaxWins`, which utilize all elements in the matrix. Let $\mathbf{M}_i^*$ and $\mathbf{M}_*^j$ denote the i-th row and j-th column of the matrix as vectors. For each candidate y_i, its `MaxLogits` score is defined as $s_i = \sum(\mathbf{M}_i^* - \mathbf{M}_*^i)$, while its `MaxWins` score is defined as $s_i = |\{s_{ij} \in \mathbf{M}_i^* | s_{ij} > 0\}| + |\{s_{ji} \in \mathbf{M}_*^i | s_{ji} < 0\}|$, where $||$ denotes the set size.

In essence, `MaxLogits` computes the confidence that y_i is superior to all other candidates, whereas `MaxWins` calculates the number of victories in comparisons with other candidates.

However, these two methods necessitate $O(N^2)$ iterations for N candidates, which can be computationally burdensome. Thus, we propose a more efficient aggregation method, performing *a single bubble sort run* with pairwise comparisons to select the best candidate. We first shuffle the order of candidates in $\mathbb{Y}$ to obtain a default order, and initialize the best candidate index k to 1. We iteratively update the best candidate index as follows:

$$k = \begin{cases} k, & \mathbf{M}_k^i - \mathbf{M}_i^k > 0 \\ i, & \mathbf{M}_i^k - \mathbf{M}_k^i > 0 \end{cases}.$$

After $N-1$ comparisons, we select y_k as the best candidate. This method reduces the inference time complexity from $O(N^2)$ to $O(N)$, aligning with previous pointwise methods.

Regardless of the aggregation method, we can rank all candidates in $\mathbb{Y}$. Our experiments (shown in the appendix) reveal that `MaxLogits` yields the best performance, so we use `MaxLogits` as the default aggregator for PairRanker.

4 GenFuser: Generative Fusion

The effectiveness of PairRanker is constrained by the quality of selections from the candidate pool $\mathbb{Y}$. We hypothesize that by merging multiple top-ranked candidates, we can overcome this

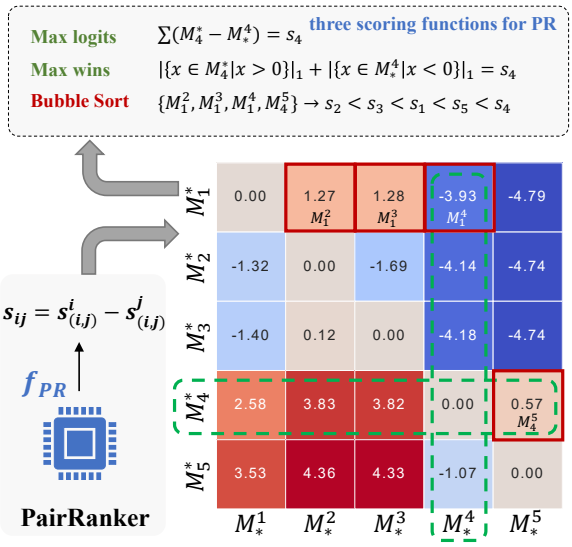

Figure 4: **Aggregation methods for PairRanker.**

constraint. As these top candidates often showcase complementary strengths and weaknesses, it is plausible to generate a superior response by combining their advantages while mitigating their shortcomings. Our objective is to devise a generative model that takes input x and K top-ranked candidates $\{y_1, ..., y_K\} \subset \mathbb{Y}$ (e.g., $K = 3$) and produces an improved output $\hat{y}$ as the final response.

To accomplish this, we present GenFuser, a seq2seq approach for fusing a set of candidates conditioned on the input instruction to generate an enhanced output. Specifically, we concatenate the input and K candidates sequentially using separator tokens, such as `<extra_id_i>`, and fine-tune a T5-like model to learn to generate y. In practice, we employ Flan-T5-XL (Chung et al., 2022), which has 3b parameters, due to its superior performance and relatively smaller size.

5 Evaluation

5.1 Setup

We use `MixInstruct` (Sec. 2.2) to conduct evaluation, and more results are in the appendix.

NLG metrics. We employ two types of evaluation metrics (i.e., Q). The first group is conventional automatic metrics for NLG tasks: BERTScore (Zhang et al., 2020b), BLEURT (Sellam et al., 2020), and BARTScore (Yuan et al., 2021).

GPT-Rank. The second is based on prompting ChatGPT for pairwise comparisions on all candidates and decide their rank by the number of wins

Category	Methods	BERTScore↑	BARTScore↑	BLEURT↑	GPT-Rank↓	≥ Vic(%)↑	≥ OA(%)↑	Top-3(%)↑
LLMs	Open Assistant (LAION-AI, 2023)	**74.68**	-3.45	**-0.39**	**3.90**	**62.78**	N/A	51.98
	Vicuna (Chiang et al., 2023)	69.60	**-3.44**	-0.61	4.13	N/A	**64.77**	**52.88**
	Alpaca (Taori et al., 2023)	71.46	-3.57	-0.53	4.62	56.70	61.35	44.46
	Baize (Xu et al., 2023)	65.57	-3.53	-0.66	4.86	52.76	56.40	38.80
	MOSS (Sun and Qiu, 2023)	64.85	-3.65	-0.73	5.09	51.62	51.79	38.27
	ChatGLM (Du et al., 2022)	70.38	-3.52	-0.62	5.63	44.04	45.67	28.78
	Koala (Geng et al., 2023)	63.96	-3.85	-0.84	6.76	39.93	39.01	22.55
	Dolly V2 (Conover et al., 2023)	62.26	-3.83	-0.87	6.90	33.33	31.44	16.45
	Mosaic MPT (MosaicML, 2023)	63.21	-3.72	-0.82	7.19	30.87	30.16	16.24
	StableLM (Stability-AI, 2023)	62.47	-4.12	-0.98	8.71	21.55	19.87	7.96
	Flan-T5 (Chung et al., 2022)	64.92	-4.57	-1.23	8.81	23.89	19.93	5.32
Analysis	Oracle (BERTScore)	**77.67**	-3.17	-0.27	3.88	54.41	38.84	53.49
	Oracle (BLEURT)	75.02	-3.15	**-0.15**	3.77	55.61	45.80	55.36
	Oracle (BARTScore)	73.23	**-2.87**	-0.38	3.69	50.32	57.01	57.33
	Oracle (GPT-Rank)	70.32	-3.33	-0.51	**1.00**	**100.00**	**100.00**	**100.00**
Rankers	Random	66.36	-3.76	-0.77	6.14	37.75	36.91	29.05
	MLM-Scoring	64.77	-4.03	-0.88	7.00	33.87	30.39	21.46
	SimCLS	**73.14**	-3.22	-0.38	3.50	52.11	49.93	60.72
	SummaReranker	71.60	-3.25	-0.41	3.66	**55.63**	48.46	57.54
	PairRanker	72.97	**-3.14**	**-0.37**	**3.20**	54.76	**57.79**	**65.12**
LLM-BLENDER	**PR** (K = 3) **+ GF**	**79.09**	**-3.02**	**-0.17**	**3.01**	**70.73**	**77.72**	**68.59**

Table 2: Empirical results on MixInstruct. GPT-Rank are the most important metric.

(i.e., MaxWins aggregation). We name this GPT-based ranking metric with GPT-Rank.

Model training. We use the DeBERTa (He et al., 2021) (400m) as the backbone for PAIRRANKER, and GENFUSER is based on Flan-T5-XL (3b). According to our ablation studies, we choose to use BARTScore for its superior correlation with GPT-Rank as shown in 5.2.

5.2 Main results

In Table 2, we present the overall performance of N=11 LLMs as well as other methods on MixInstruct. In addition to the three auto metrics and GPT-Rank, we also show the percentage of examples where each method can produce outputs that are *better than or same good* as the two top LLMs, namely OpenAssistant (≥OA) and Vicuna (≥Vic), in terms of GPT-Rank.

LLMs have diverse strengths and weakness. The table presents the LLMs in a sorted order based on their average rank as determined by ChatGPT (GPT-Rank). Among these models, Open Assistant, Vicuna, and Alpaca are the top-3 performers. Following them, three renowned LLMs, namely Baize, Moss, and ChatGLM, which have been fine-tuned using both Chinese and English instruction data, also exhibit impressive performance on MixInstruct. Conversely, Mosaic MPT, StableLM, and Flan-T5 rank at the bottom-3 in the evaluation. Nevertheless, the average GPT-Rank of top/bottom models maintain a noticeable distance from the first/last position (1 or 11), highlighting the importance of ensembling LLMs.

Top LLMs are not always good. It is evident that although OA and Vic perform remarkably well, there is still a substantial percentage of examples where other LLMs are considered to outperform them. For instance, despite Koala having an average GPT-Rank of 6.76, approximately 40% of the examples demonstrate that Koala produces responses that are better or equally as good as both OA and Vic. This further emphasizes the significance of employing our LLM-BLENDER framework for ranking and fusion purposes.

NLG Metrics. Moreover, we conduct a comprehensive analysis of the performance of oracle (top-1) selections based on each of the metrics themselves. The findings demonstrate that these selections also exhibit favorable performance across other metrics as well. For example, the oracle selections derived from **GPT-Rank** achieve a BARTScore of −3.33, surpassing that of OA (−3.45). Conversely, the oracle selections of BARTScore yield 3.69 in GPT-Rank, also significantly outperforming OA (3.90). This observation substantiates the rationality of using BARTScore to provide supervision for PAIRRANKER, which is also suggested by Table 3.

PAIRRANKER outperforms other rankers. MLM-Scoring fails to outperform even random selection, highlighting the limitations of its unsupervised paradigm. On the contrary, SimCLS, SummaReranker, and PAIRRANKER exhibit su-

Ranking Methods	Pearson Correlation ↑	Spearman's Correlation ↑	Spearman's Footrule ↓
Random	0.00	0.00	48.27
BLEU	28.70	26.92	33.57
Rouge2	29.17	27.77	32.96
BERTScore	32.25	30.33	33.34
BLEURT	34.14	32.31	32.17
BARTScore	**38.49**	**36.76**	**30.93**
MLM-Scoring	-0.02	-0.01	47.16
SimCLS	39.89	38.13	29.32
SummaReranker	41.13	39.10	29.69
PairRanker	**46.98**	**44.98**	**27.52**

Table 3: The correlation between each ranking method and oracle ranking (GPT-Rank).

perior performance compared to the best model (OA) across BARTScore and GPT-Rank. Notably, the average GPT-rank of the responses selected by PAIRRANKER (3.20) significantly outperforms the best model by 0.70 (a 18% relative performance gain) and also all other rankers. Moreover, it achieves impressive results in metrics such as BARTScore (−3.14) with a substantial advantage. PAIRRANKER's selections are better than or equal to Vic/OA on 54.76%/57.79% examples respectively, and ranks in top 3 for 65.12% examples.

LLM-BLENDER is the best. We use top-3 selections from the PAIRRANKER and feed them as candidates for GENFUSER. Based on this integration, LLM-BLENDER demonstrates remarkable capabilities as expected. In terms of GPT-Rank, it achieves 3.01, surpassing both the best model OA (3.90) by a significant margin. The scores for BERTScore (79.09), BARTScore (−3.02), and BELURT (−0.17) all exceed the best model by 4.41, 0.43, and 0.22 respectively, showcasing substantial advantages. Moreover, LLM-BLENDER also performs well in surpassing the top two models, Vic (70.73) and OA (77.72), thereby complementing the weaknesses of PAIRRANKER.

Ranking correlation. In addition to focusing solely on the top-1 selection of each ranker, we present a comprehensive analysis of the overall rank correlation among all the candidates with GPT-Rank (see Table 3). The correlation metrics used here include the Pearson Correlation Coefficient, Spearman's Correlation, and Spearman's Footrule distance(Diaconis and Graham, 1977).

It turns out that BARTScore gets the highest correlation with GPT-Rank against other metrics, which suggests we use BARTScore to provide supervision for training. For rankers, MLM-Scoring still falls short of outperforming random permutations. On the other side, SummaReranker demonstrates better correlation in terms of the Pearson Correlation (41.13) and Spearman's Correlation (39.10), while SimCLS gets a better Spearman's Footrule distance (29.32) Notably, PAIRRANKER achieves the highest correlation with GPT-Rank across all correlation types, which is even way better than the BARTScore.

More analysis. We leave many other ablation studies and analyses in Appendix, where we apply PAIRRANKER to the three typical natural language generation (NLG) tasks: summarization (CNN/DM), machine translation (WMT18-zh-en), and constrained text generation (CommonGen). We find that PAIRRANKER still outperforms other methods by a large margin in the context of using a single same base model to decode N candidates (with different algorithms). We also show that MaxLogits is much better than MaxWins and the bubble sort method is very cost-effective if the inference efficiency is a big concern.

6 Related Work

LLM evaluation As open-source large language models (LLMs) continue to flourish and demonstrate remarkable competitiveness across various natural language generation (NLG) tasks, assessing the capabilities of LLMs has become an exceedingly challenging endeavor. To address this issue, Zheng et al. (2023) pioneered the creation of a chatbot arena, enabling users to provide pairwise evaluations of responses generated by two randomly selected LLMs. Based on these evaluations, they established an LLM Elo rating leaderboard. In a similar vein, Cabrera and Neubig (2023) conducted an evaluation study on a customer service dataset, leveraging automated metrics such as BERTScore and ChrF (Popović, 2015). This approach yielded similar LLM ranking results. Instead of relying solely on human evaluation, Yidong et al. (2023) developed a fine-tuned model called PandaLM to compare responses generated by different LLMs. AlpacaFarm (Dubois et al., 2023) also aims to evaluate LLMs with pairwise feedback.

Pairwise ranking Pairwise ranking, known for its long-standing effectiveness, has demonstrated exceptional performance across a wide array of NLP tasks (Jamieson and Nowak, 2011). Notably, Ranknet (Burges et al., 2005) and Lamb-

daRank (Burges, 2010) have emerged as powerful techniques for various ranking problems. Furthermore, within the renowned RLHF procedure(Ouyang et al., 2022), these methods incorporate pairwise training of their reward model based on OPT. However, these approaches still compute scores individually and solely undergo pairwise training at the loss level. In contrast, our proposed PAIRRERANKER not only employs pairwise training but also utilizes the attention mechanism for pairwise inference during the inference stage. We posit that this approach better captures the subtleties between candidates and yields superior results, as demonstrated in Section 5.2.

Ensemble learning Ensemble learning is a widely employed technique to enhance a model's capabilities by leveraging multiple weaker models (Sagi and Rokach, 2018; Anioł and Pietroń, 2019). Typically, ensemble learning is performed either by considering model weights or by combining diverse outputs. Mix-of-Experts (MoE) is a type of ensemble approach that combines the predictions of multiple specialized sub-models to improve overall performance. It has been successfully applied in various domains, such as natural language processing and computer vision (Jacobs et al., 1991; Shazeer et al., 2017). As for fusing multiple candidates, Izacard and Grave (2021) introduced a framework named Fusion-in-Decoder (FiD) to improve the quality of question answering by fusing retrieved text. Building upon FiD, Ravaut et al. (2022b) further investigated the effectiveness of fusion in the context of text summarization. However, they neglected to incorporate a selection process prior to feeding the candidates into the fusion module, resulting in only moderate improvements. In contrast, our proposed approach, referred to as LLM-BLENDER, initially utilizes the PAIRRANKER algorithm to filter out candidates of poor quality. Subsequently, fusion is performed exclusively on the top-ranked candidates, leading to superior performance.

7 Conclusion & Future Directions

In this paper, we formulated the motivation to exploit the diverse strengths and weaknesses of open-source large language models (LLMs), aiming to create an ensembling framework that leverages their complementary capabilities to generate consistently superior results on various instruction-following tasks. By dynamically ensembling LLMs, we aimed to reduce biases, errors, and uncertainties in individual models, yielding outputs better aligned with human feedback.

Our major contributions are as follows:

- **A new framework:** **LLM-BLENDER** is a post-hoc ensemble learning method for ranking and fusing the outputs from multiple LLMs. It is composed of two modules: PAIRRANKER and GENFUSER, and both are straightforward yet effective.
- **A new dataset:** **MixInstruct** is a benchmark dataset, created for training and evaluating LLM ensembling methods on instruction-following tasks.
- **Promising results:** We show that our method can significantly improve the overall results on various metrics, and our findings indicates that this direction is promising for both research community and practitioners.
- **Toolkit:** By open-sourcing our framework, we aim to make it easier for others to leverage our approach, enabling the development of more advanced AI systems that achieve robustness, generalization, and enhanced accuracy in a wide variety of tasks.

Future directions. Potential future directions include extending the LLM-BLENDER framework to more types of models or even non-text modalities, developing more sophisticated ranking and fusion techniques, and investigating the transferability of our ensembling approach to other domains and tasks. Additionally, exploring ways to minimize computational overhead and incorporating active learning strategies for rapid adaptation to new specialized domains and data sources represent fruitful areas for further research. Overall, our work underscores the value of combining the unique contributions of multiple models.

*Limitations

Efficiency. To get the optimal performance from PAIRRANKER, one may need to call the model $O(n^2)$ times for getting the full matrix, thus resulting in a much less efficient solution. We attempted to resolve this limitation by proposing to use multiple rounds of bubble sort methods to reduce the number of inferences needed, and we find it works pretty well. We also want to argue that although

the number of inferences can be large for obtaining the best performance with PAIRRANKER, those inferences can be executed in parallel because they are totally independent.

Human evaluation. We agree that automatic metrics have limitations. Human evaluation could provide us with more reliable and comprehensive evaluation results. However, due to the number of models as well as the amounts of generation candidates, we cannot afford large-scale human evaluation. We argue that our use of ChatGPT for evaluation is a good alternative, according to recent studies. Also, we would like to highlight that we show the ground truths when using ChatGPT to do pairwise comparisions, which is quite informative than the common practice.

*Ethical Statement

This work fully complies with the ACL Ethics Policy. We declare that there are no ethical issues in this paper, to the best of our knowledge.

Acknowledgements

We thank members of the INK lab at USC and the Mosaic team at AI2 for valuable feedback on this project. Xiang is supported in part by the Office of the Director of National Intelligence (ODNI), Intelligence Advanced Research Projects Activity (IARPA), via the HIATUS Program contract #2022-22072200006, the DARPA MCS program under Contract No. N660011924033, the Defense Advanced Research Projects Agency with award W911NF-19-20271, NSF IIS 2048211, and gift awards from Google and Amazon. Yuchen's research was also supported by the Allen Institute for AI (AI2). The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of ODNI, IARPA, or the U.S. Government.

References

Anna Anioł and Marcin Pietroń. 2019. Ensemble approach for natural language question answering problem. *2019 Seventh International Symposium on Computing and Networking Workshops (CANDARW)*, pages 180–183.

Stella Rose Biderman, Hailey Schoelkopf, Quentin G. Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A suite for analyzing large language models across training and scaling. *ArXiv preprint*, abs/2304.01373.

Ondřej Bojar, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Shujian Huang, Matthias Huck, Philipp Koehn, Qun Liu, Varvara Logacheva, Christof Monz, Matteo Negri, Matt Post, Raphael Rubino, Lucia Specia, and Marco Turchi. 2017. Findings of the 2017 conference on machine translation (WMT17). In *Proceedings of the Second Conference on Machine Translation*, pages 169–214, Copenhagen, Denmark. Association for Computational Linguistics.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, John A. Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuan-Fang Li, Scott M. Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *ArXiv preprint*, abs/2303.12712.

Christopher J. C. Burges. 2010. From ranknet to lambdarank to lambdamart: An overview.

Christopher J. C. Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Gregory N. Hullender. 2005. Learning to rank using gradient descent. In *Machine Learning, Proceedings of the Twenty-Second International Conference (ICML 2005), Bonn, Germany, August 7-11, 2005*, volume 119 of *ACM International Conference Proceeding Series*, pages 89–96. ACM.

Alex Cabrera and Graham Neubig. 2023. Zeno chatbot report. Blog post.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier García, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta,

Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways. *ArXiv preprint*, abs/2204.02311.

Hyung Won Chung, Le Hou, S. Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Wei Yu, Vincent Zhao, Yanping Huang, Andrew M. Dai, Hongkun Yu, Slav Petrov, Ed Huai hsin Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022. Scaling instruction-finetuned language models. *ArXiv preprint*, abs/2210.11416.

Mike Conover, Matt Hayes, Ankit Mathur, Xiangrui Meng, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. 2023. Free dolly: Introducing the world's first truly open instruction-tuned llm.

Persi Diaconis and Ron Graham. 1977. Spearman's footrule as a measure of disarray. *Journal of the royal statistical society series b-methodological*, 39:262–268.

Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 320–335, Dublin, Ireland. Association for Computational Linguistics.

Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori Hashimoto. 2023. Alpacafarm: A simulation framework for methods that learn from human feedback. *ArXiv preprint*, abs/2305.14387.

Xinyang Geng, Arnav Gudibande, Hao Liu, Eric Wallace, Pieter Abbeel, Sergey Levine, and Dawn Song. 2023. Koala: A dialogue model for academic research. Blog post.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. Debertav3: Improving deberta using electra-style pretraining with gradient-disentangled embedding sharing. *ArXiv preprint*, abs/2111.09543.

Karl Moritz Hermann, Tomás Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, pages 1693–1701.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880, Online. Association for Computational Linguistics.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive mixtures of local experts. *Neural Computation*, 3:79–87.

Kevin G. Jamieson and Robert D. Nowak. 2011. Active ranking using pairwise comparisons. In *Advances in Neural Information Processing Systems 24: 25th Annual Conference on Neural Information Processing Systems 2011. Proceedings of a meeting held 12-14 December 2011, Granada, Spain*, pages 2240–2248.

LAION-AI. 2023. Open assistant. `https://github.com/LAION-AI/Open-Assistant`.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.

Bill Yuchen Lin, Wangchunshu Zhou, Ming Shen, Pei Zhou, Chandra Bhagavatula, Yejin Choi, and Xiang Ren. 2020. CommonGen: A constrained text generation challenge for generative commonsense reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1823–1840, Online. Association for Computational Linguistics.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *ArXiv preprint*, abs/1907.11692.

Yixin Liu and Pengfei Liu. 2021. SimCLS: A simple framework for contrastive learning of abstractive summarization. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 1065–1072, Online. Association for Computational Linguistics.

NLP Team MosaicML. 2023. Introducing mpt-7b: A new standard for open-source, ly usable llms. Accessed: 2023-05-23.

Ramesh Nallapati, Bowen Zhou, Cicero dos Santos, Çağlar Gulçehre, and Bing Xiang. 2016. Abstractive text summarization using sequence-to-sequence RNNs and beyond. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 280–290, Berlin, Germany. Association for Computational Linguistics.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. 2022. Training language models to follow instructions with human feedback. *ArXiv preprint*, abs/2203.02155.

Maja Popović. 2015. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal. Association for Computational Linguistics.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67.

Mathieu Ravaut, Shafiq Joty, and Nancy Chen. 2022a. SummaReranker: A multi-task mixture-of-experts re-ranking framework for abstractive summarization. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4504–4524, Dublin, Ireland. Association for Computational Linguistics.

Mathieu Ravaut, Shafiq Joty, and Nancy Chen. 2022b. Towards summary candidates fusion. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8488–8504, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Omer Sagi and Lior Rokach. 2018. Ensemble learning: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 8.

Julian Salazar, Davis Liang, Toan Q. Nguyen, and Katrin Kirchhoff. 2020. Masked language model scoring. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2699–2712, Online. Association for Computational Linguistics.

Thibault Sellam, Dipanjan Das, and Ankur Parikh. 2020. BLEURT: Learning robust metrics for text generation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7881–7892, Online. Association for Computational Linguistics.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net.

Noam Shazeer and Mitchell Stern. 2018. Adafactor: Adaptive learning rates with sublinear memory cost. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 4603–4611. PMLR.

Stability-AI. 2023. Stablelm: Stability ai language models. https://github.com/stability-AI/stableLM.

Tianxiang Sun and Xipeng Qiu. 2023. Moss. https://github.com/OpenLMLab/MOSS.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

Jörg Tiedemann and Santhosh Thottingal. 2020a. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, pages 479–480, Lisboa, Portugal. European Association for Machine Translation.

Jörg Tiedemann and Santhosh Thottingal. 2020b. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, pages 479–480, Lisboa, Portugal. European Association for Machine Translation.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aur'elien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *ArXiv preprint*, abs/2302.13971.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-instruct: Aligning language model with self generated instructions. *ArXiv preprint*, abs/2212.10560.

Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. 2023. Baize: An open-source chat model with parameter-efficient tuning on self-chat data. *ArXiv preprint*, abs/2304.01196.

Wang Yidong, Yu Zhuohao, Zeng Zhengran, Yang Linyi, Heng Qiang, Wang Cunxiang, Chen Hao, Jiang Chaoya, Xie Rui, Wang Jindong, Xie Xing, Ye Wei, Zhang Shikun, and Zhang Yue. 2023. Pandalm: Reproducible and automated language model assessment. https://github.com/WeOpenML/PandaLM.

Weizhe Yuan, Graham Neubig, and Pengfei Liu. 2021. Bartscore: Evaluating generated text as text generation. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 27263–27277.

Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter J. Liu. 2020a. PEGASUS: pre-training with extracted gap-sentences for abstractive summarization. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 11328–11339. PMLR.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020b. Bertscore: Evaluating text generation with BERT. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena.

Appendix

A Implementation Details

PAIRRANKER We train our ranker for 5 epochs. We use the Adafactor optimizer (Shazeer and Stern, 2018), with the maximum learning rate being 1e-5. The warm-up ratio is 5% with a linear learning rate scheduler. Our training batch size is 64. The training finishes on a single RTX 8000 GPU in two days. The backbone of PAIRRANKER is Deberta-v3-large (He et al., 2021). Unlike the mixture-of-experts layer used in the work of Ravaut et al. (2022a), we employ a five-layer multi-layer perceptron (MLP) with the hyperbolic tangent activation function. The output dimension of the final layer is equal to the number of different metrics. In practice, we have tried different special embedding combinations, such as only feeding `<candidate1>` and `<candidate2>`, mean pooling representation, and so on. And Finally, we found that concatenating `<source>` with `<candidate1>`, and `<source>` with `<candidate2>` respectively achieves the best performance. We also tried different loss types, like MSE losses and ranking losses. And we find BCE is simply good enough.

GENFUSER We use Flan-T5-large and Flan-T5-xl (3b) to train the top-3 bart-score selections as input and then apply it with PAIRRANKER's top 3 selections for inference. We find Flan-t5-3b performs much better than the large version, while flan-t5-xxl has marginal improvements yet being much larger and longer to train.

B Conventional Tasks

To quantitatively understand how sub-optimal the default selections of decoding methods are, we present an empirical analysis in Fig. 5 . Here we look at three typical NLG tasks: summarization (CNN/DM), machine translation (WMT18), and constrained text generation (CommonGen), with their popularly used base models: PEGASUS (Zhang et al., 2020a), Opus-MT (Tiedemann and Thottingal, 2020a), and T5-large (Raffel et al., 2020). We can see that the default selections (yellow bars in Fig. 5; the top-beam generations) are much worse than the oracle selections from the top 15 candidate generations for each decoding method (blue bars).

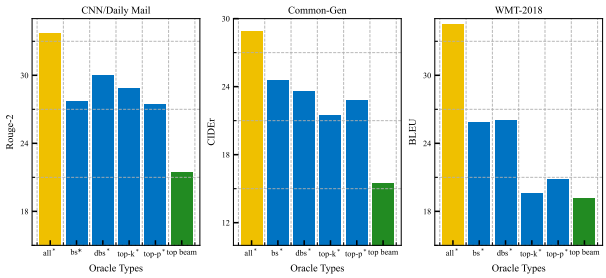

Figure 5: The comparisions between different decoders and oracle selections.

Moreover, if we combine the results from the four methods as a larger candidate pool, then the performance (green bars) of these NLG models can be much improved. For example, the Rouge-2 score of PEAGUS can be improved by 57% and the BLEU score for Opus-MT can be improved by nearly 80%, compared to their top-beam performance. Simply put, the default selections (i.e., the generations with the highest decoding scores) are much worse than the best selections from a relatively small candidate pool. Therefore, we argue that it is of significant importance to rerank generation candidates in order to enhance the performance of LMs in NLG tasks.

Why do decoding algorithms often overlook generation candidates of better quality? The lower quality of default selections is often attributed to the exposure bias caused by the teacher-forcing paradigm in most auto-regressive models. Plus, the greediness of the search process and the randomness in sampling are also part of the reasons. Re-ranking has been a simple yet effective post hoc approach to mitigate this gap. For instance, MLM-scoring (Salazar et al., 2020) uses an external LM such as BERT to estimate the quality of a candidate without any supervision. SimCLS (Liu and Liu, 2021) trains a re-ranker using a simple contrastive training objective, which encodes the source text and each candidate output using the same encoder and scores each candidate based on the cosine similarity between the embeddings. Another successful approach is SummaReranker (SR) (Ravaut et al., 2022a), which is trained to improve the re-ranker for multiple metrics simultaneously.

C Additional Results on Conventional Tasks

In this section, we evaluate the PAIRRANKER with conventional natural language generation (NLG)

tasks. Extensive experiments conducted on three NLG tasks (i.e., summarization, translation, and constrained sentence generation) demonstrate that PairRanker outperforms the baseline methods by a consistent margin and is also compatible with very large language models such as GPT-3 (text-davinci-003). PairRanker not only outperforms the previous state-of-the-art method SummaReranker on the summarization task, but also shows great generalization performance in the other two NLG tasks, which are not evaluated previously. In addition, our PairRanker can be transferred to improve GPT-3 results by 26.53% and 11.65% for CommonGen and WMT18 (zh-en) respectively, even though our rerankers are not trained with any candidates decoded by GPT-3 models.

Method ↓ Metric →	R-1	R-2	R-L	Gain_{R1}
BART	44.48	21.21	41.60	-
PEGASUS	44.56	20.90	41.58	-
Gsum	45.94	22.32	42.48	-
Gsum+RefSum	46.18	22.36	42.91	1.18%
BART+SimCLS	46.67	22.15	43.54	4.92%
PEGASUS+MLM-Scoring	43.03	19.48	40.12	-3.43%
PEGASUS+SummaReranker	47.16	22.55	43.87	5.83%
PEGASUS+**PairReranker** (bubble)	47.29	22.77	44.06	6.13%
PEGASUS+**PairReranker** (max wins)	47.29	22.79	44.07	6.13%
PEGASUS+**PairReranker** (max logits)	**47.39**	**22.91**	**44.18**	**6.35%**
GPT-3 (text-davinci-003)	37.96	15.51	34.39	-
GPT-3-oracle	45.46	22.83	42.04	19.76%
GPT-3+MLM-Scoring	38.13	15.09	34.32	0.45%
GPT-3+SummaReranker	39.62	17.13	36.12	4.37%
GPT-3+**PairReranker** (bubble)	40.41	17.44	36.79	6.45%
GPT-3+**PairReranker** (max wins)	40.37	17.46	36.76	6.35%
GPT-3+**PairReranker** (max logits)	**40.48**	**17.54**	**36.84**	**6.64%**

Table 4: Model performance on **CNN/DailyMail**.

C.1 Tasks and data creation

We evaluate reranking methods on the following public dataset: CNN/DM, CommonGen, and WMT18 (zh-en). The data statistics of these benchmarks are in Table 7 (in Appendix).

CNN/DM (Hermann et al., 2015) is a dataset constructed from CNN and DailyMail websites. It is first used for machine-reading and comprehension, and later Nallapati et al. (2016) use it for abstractive summarization. Evaluation metrics are Rouge-1, Rouge-2, and Rouge-L.

CommonGen (Lin et al., 2020) is a dataset used for generative commonsense reasoning. It contains 79K commonsense descriptions where the language model is required to compose a realistically plausible sentence from given concepts. Evaluation metrics are BLEU and CIDEr.

WMT2018 (Bojar et al., 2017) is a well-known dataset for evaluate machine translation. Here we use the Chinese-English split for evaluation. Evaluation metrics are BLEU.

C.2 Base models

For the summarization task on CNN/DailyMail dataset, we use the famous PEGASUS-large (Zhang et al., 2020a) and BART-large (Lewis et al., 2020), which have exhibited great ability for abstractive summarization. We use the public fine-tuned checkpoint from Hugging face. For the generative commonsense reasoning task on CommonGen dataset, we use the T5-large (Raffel et al., 2020). It's one of the vanilla baselines reported in Lin et al. (2020). For the Chinese-English translation task on WMT18 dataset, we use the public pre-trained opus-mt checkpoint (Tiedemann and Thottingal, 2020b).

C.3 Evaluation setups

In this section, we talk about the training and testing paradigm of our reranker, including how we construct the training, validation, and testing dataset for our reranker, how we generate candidates, and what our experiment focuses on.

To construct the training dataset for the reranker, we need to ensure the base model used to generate candidates on the training dataset should never have seen these candidates. Otherwise, the reranker will be trained on the candidates with higher quality compared to the candidates that it will be tested on, which we found will result in fairly bad performance. Therefore, following Ravaut et al. (2022a), we first fine-tune the original non-finetuned pre-trained model on half of the training dataset, which gives us 2 half-finetuned base models that each of them has only seen their own half of the training dataset. Then we use them to generate candidates on their un-seen half of the training dataset using the decoding method talked about before. These generated candidates together form a whole training dataset with generated candidates that resemble the quality during the inference stage.

During the inference stage, we directly adopt the public checkpoints that have been finetuned on the whole training dataset. We generate candidates on the validation and test datasets with this public checkpoint, which constitutes the validation and testing datasets on which our reranker runs inference. We use two decoding methods, beam search,

Method ↓ Metric →	BLEU	CIDEr	$\text{Gain}_{\text{CIDEr}}$
T5-large	14.62	15.48	-
T5-large+MLM-Scoring	14.04	14.12	-8.79%
T5-large+SimCLS	14.5	14.99	-3.17
T5-large + SummaReranker	14.13	15.29	-1.23%
T5-large + **PairReranker** (bubble)	15.30	**15.93**	**2.91%**
T5-large + **PairReranker** (max wins)	15.29	15.91	2.78%
T5-large + **PairReranker** (max logits)	**15.40**	15.86	2.45%
GPT-3 (text-davinci-003)	11.85	11.12	-
GPT-3 + oracle	20.34	19.26	73.20%
GPT-3 + MLM-Scoring	12.56	11.66	4.86%
GPT-3 + SummaReranker	13.71	13.21	18.79%
GPT-3 + **PairReranker** (bubble)	14.39	13.85	24.55%
GPT-3 + **PairReranker** (max wins)	14.32	13.76	23.74%
GPT-3 + **PairReranker** (max logits)	**14.63**	**14.07**	**26.53%**

Table 5: Model performance on **CommonGen**.

Method ↓ Metric →	BLEU	Gain
Opus-MT	19.29	-
Opus-MT+MLM-Scoring	16.35	-15.24%
Opus-MT+SimCLS	18.93	-1.87%
Opus-MT+SummaReranker	19.40	0.57%
Opus-MT+**PairReranker (bubble)**	20.36	5.54%
Opus-MT+PairReranker (max wins)	20.30	5.24%
Opus-MT+**PairReranker (max logits)**	**20.47**	**6.12%**
GPT-3 (text-davinci-003)	23.61	-
GPT-3 + oracle	36.11	52.94%
GPT-3+MLM-Scoring	23.98	1.57%
GPT-3+SummaReranker	25.08	6.22%
GPT-3+**PairReranker** (bubble)	26.29	11.35%
GPT-3+**PairReranker** (max wins)	**26.36**	**11.65%**
GPT-3+**PairReranker** (max logits)	26.19	10.93%

Table 6: Model performance on **WMT18 (zh-en)**.

and diverse beam search, in the experiments, following the prior work of SummaReranker. We generate 15 candidates for each decoding method for both training and inference.

C.4 Main results

Overall performance in summarization. Following the training and testing paradigm stated in section C.3, we briefly report the test results on the CNN/DM dataset in Tab. 4. With fine-tuned PEGASUS-large as the base model. our *Max Logits* method improves the candidates' quality by 6.35% in Rouge-1, which is higher than our baseline SummaReranker. Besides, the performance gains in other metrics like Rouge-2 (9.62%) and Rouge-L (6.25%) are also obviously better.

Can PairReranker generalize to other generation tasks? Yes. In order to test the task generalization ability of our method, we here report the test results on CommonGen and WMT2018 (zh-en) in Tab. 5 and Tab. 6. From the data in the table, our method also improves the candidates' quality significantly after reranking. Our *Max Logits* method obtain a 2.45% performance gain in CIDEr on the CommonGen dataset and a 6.12% performance gain in BLEU on the WMT2018 dataset. What's more, it's worth noting our *bubble run* method achieves an even higher gain in CIDEr (2.91%).

We also report the performance of SummaReranker on the two datasets. In contrast to the great performance on summarization, SummaReranker seems to fail to generalize well on other datasets. We also find that SummaReranker obtains a decreased gain on the CommonGen dataset (-1.23% in CIDEr). The improvement on the translation dataset is not obvious (0.57% in BLEU). We hypothesize that this is because of the average length of the candidates and the target text in these two datasets are all significantly smaller than the one in summarization (see in Tab. 7). Therefore, the higher in-group similarity brought by the shorter length makes it harder for SummaReranker to capture their difference. On the contrary, our method with direct attention between a pair of candidates could easily tackle this problem.

Can PairReranker generalize other large-scale models like GPT-3? Yes. After training on an expert dataset, our reranker could directly be applied to other models' outputs selection for the same task. To support this, we directly apply our three rerankers trained on the 3 main tasks respectively to the GPT-3 outputs with proper task-specific prompts. We report the performance gain in Tab. 4, 5, and 6. From the data reported in the table, we could see that the quality of the GPT-3 outputs is improved by a large margin compared to the average. Also, our performance gain is significantly larger than the baseline SummaReranker. For example, on the GPT-3 data points sampled from CNN/DM, our max logits method obtain a gain of 6.64%, whereas SummaReranker only obtains a gain of 4.37%. And on the CommonGen's, our method obtains a gain of 26.53% and SummaReranker only obtains a gain of 18.79%.

Can I make trade-offs between performance and number of comparisons? Yes. Due to the high cost of full comparison methods, it's necessary for us to study the trade-off between the model performance and the number of comparisons. For full comparison methods, we first initialize matrix M in Figure 4 to be all zeroes, Then every time of the comparison, we fill a confidence cell that is zero

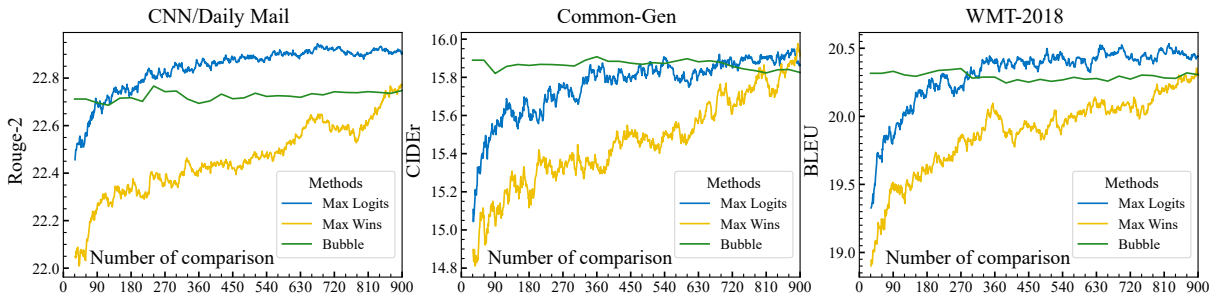

Figure 6: Efficiency trade-off with the number of pairwise comparisons

before, then do the scoring and select the best one based on the current information in the matrix. For bubble run, we run multiple times of bubble run and select one that is chosen as the best one for the most times. Each bubble cost N comparisons. The trade-off results are shown in Figure 6. From the results, we could see bubble run method could achieve high performance with little cost. However, as the number of comparisons increases, *Max Logits* scoring methods will surpass the bubble run after a certain number of comparisons. We contend that the bubble run method already reports a pretty good performance with $N-1$ times of comparison. Therefore, most of the time, bubble run is a more efficient way to apply. If you want to pursue the marginal improvements brought by more comparison, you can also apply *Max Logits* method with parallel computing.

C.5 Model Further Study

Due to the order of the input format, changing the position of candidate 1 and candidate 2 might also change the results (Sec. 3.3). In practice, we found that by simply shuffling the order of candidate 1 and candidate 2, our reranker could be consistent with itself more than 90% of the time.

We analyze the model's relation between consistency as well as accuracy and the absolute pair rank difference. The results are presented in Figure 7. From the results, we could see that the model is better at classifying candidates with a higher absolute rank difference, because they are supposed to be more different.

D Dataset statistics

We analyze the basic statistics, including the number of examples and the average words per example, of the 3 datasets. The data is presented in Table 7

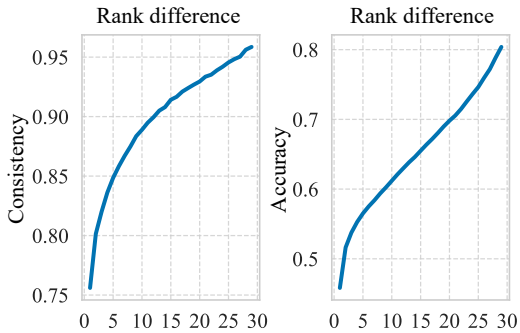

Figure 7: Consistency and accuracy analysis for CNN/Daily Mail Dataset

Dataset	# Examples			# Words per example	
	Train	Val	Test	Source	Target
CNN/DM	287k	13k	11,490	856.56	70.05
CommonGen	67k	4k	1,497	4.20	12.92
WMT18(zh-en)	25m	2k	3,981	83.48	30.95

Table 7: Statistics of the three datasets.

E ChatGPT Comparison Prompting Template (GPT-Rank)

Template	Instruction: ${instruction} Input: ${input} Candidate A: ${candidate1} Candidate B: ${candidate2} Given the instruction and input above, please compare the two candidates. You only have 4 choices to output: If you think A is better, please output: 1. A is better If you think B is better, please output: 2. B is better If you think both are good enough correctly give the answer, please output: 3. Same good If you think both are bad and do not follow the instruction, please output: 4. Same bad Do not output anything else except the 4 choices above. Output your choice below:
Comparison Option	1. A is better 2. B is better 3. Same good 4. Same bad

Table 8: The template used for ChatGPT comparison ranking (GPT-Rank).